Campus Recruitment Test - Phase 2: Assessment Test

Name: Akshat Modi
Batch: B1
Roll No: 36

SECTION A — Spring Boot & Request Lifecycle

Q1)
Ans:
Output:
Constructor called
Init called

Reason:

1. Constructor called - Constructor executes when Spring creates the bean object.
2. Init called - @PostConstruct method runs after dependency injection and bean initialization.

Q2)
Ans:
Mistake - return new ResponseEntity<>(HttpStatus.OK);
Corrected - return new ResponseEntity<>(u, HttpStatus.OK);

Q4)
Ans:
Annotation - @Transactional

Reason -

@Transactional ensures all database operations execute as a single transaction. If it is missing and one save operation fails, partial updates may occur causing inconsistent account balances.

Q6)

Ans:

a) This JWT filter extracts the token from the Authorization header and validates the user. If the token is valid, it sets authentication in Spring Security context.
b) Corrected line: String token = header.substring(7);

SECTION B — JPA, SQL & Data Layer

Q7)

Ans:

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

   @Query("SELECT p FROM Product p WHERE p.price < :price AND
p.category = :category")
   List<Product> findByPriceLessThanAndCategory(
         @Param("price") double price,
         @Param("category") String category
   );
}
```

Q8)

Ans:

The N+1 query problem happens when JPA first fetches a list of parent entities using 1 query, and then executes one extra query for each parent entity to fetch related child data.

Simple Example

Suppose we have:

```
@Entity
class Department {
   @OneToMany(mappedBy = "department")
   List<Employee> employees;
}
```

If we fetch all departments:

```
List<Department> list = departmentRepo.findAll();
```

JPA runs:

- 1 query to fetch all departments
- N additional queries to fetch employees of each department

So if there are 5 departments → total queries = 1 + 5 = 6 queries.

Most Common Cause

The most common cause is lazy loading (FetchType.LAZY) on relationships.

Fix using JOIN FETCH:

```
@Query("SELECT d FROM Department d JOIN FETCH d.employees")
```

Q9)

Ans:

SQL Query-

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

JPQL @Query Method in JPA Repository -

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT MAX(e2.salary) FROM Employee e2)")
    Double findSecondHighestSalary();
}
```

Q10)

(a) Find by city

```
List<Student> findByCity(String city);
```

(b) Find by name and city

```
List<Student> findByNameAndCity(String name, String city);
```

(c) Find by email containing a keyword

```
List<Student> findByEmailContaining(String keyword);
```

(d) Count students in a city

```
long countByCity(String city);
```

Q11)

Ans:

Problem Name

- This problem is called the N+1 Query Problem.
- Here, JPA first runs 1 query to load all orders, then runs 1 extra query for each order to load its items.

  So for 10 orders:

  •1 query for orders

  •10 queries for items

  Total = 11 queries.

Line of Code That Causes It

- System.out.println(o.getItems().size());

This triggers a separate query for each order because `items` are lazily loaded.

One-Line Fix

- @Query("SELECT o FROM Order o JOIN FETCH o.items")

Section C — Microservices & System Design

Q12)
Ans:
application.yml

```
spring:
  application:
    name: product-service

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Main Class Annotation

```
@EnableEurekaClient

@SpringBootApplication

public class ProductServiceApplication {

    public static void main(String[] args) {

        SpringApplication.run(ProductServiceApplication.class, args);

    }

}
```

Q13)

Ans:

A Circuit Breaker is a design pattern in microservices that prevents repeated calls to a failing service.

It is needed to:

- Avoid cascading failures in distributed systems
- Improve fault tolerance
- Return fallback responses when a service is down
- Prevent system overload and improve response time

Spring Boot Example using Resilience4j

```
@Service
public class PaymentServiceClient {
    @Autowired
    private RestTemplate restTemplate;
    @CircuitBreaker(name = "paymentService", fallbackMethod =
"fallbackPayment")
    public PaymentResponse getPaymentDetails() {
        return restTemplate.getForObject(
                "http://payment-service/payments",
                PaymentResponse.class
        );
    }
    public PaymentResponse fallbackPayment(Exception ex) {
```

```
        PaymentResponse response = new PaymentResponse();
        response.setStatus("Payment Service Unavailable");
        response.setAmount(0);

        return response;
    }
}
```

Q14)

Ans:

An API Gateway is a single entry point for all client requests in a microservices architecture. It routes requests to the appropriate microservice.

Two Benefits of Using an API Gateway

1. Centralized Routing – Handles routing of requests to different services from one place.
2. Security & Monitoring – Can manage authentication, logging, rate limiting, etc.

application.yml for Spring Cloud Gateway:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
```

```
      uri: lb://order-service
      predicates:
        - Path=/api/orders/**

    - id: user-service
      uri: lb://user-service
      predicates:
        - Path=/api/users/**
```

Q15)

Ans:

Service Discovery is a mechanism in microservices where services automatically register themselves and discover other services dynamically.

It is needed because IP addresses and ports of services can change frequently. Hardcoding them makes the system difficult to maintain and scale.

Tool used: Eureka Server

RestTemplate Bean Configuration:

```
@Configuration
public class AppConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    } }
```

Q16)

Ans:

(a) Three Benefits of Splitting into Microservices

1. Independent Deployment – Each service can be developed and deployed separately.
2. Better Scalability – Individual services can scale based on demand.
3. Technology Flexibility – Different services can use different technologies/databases if needed.

(b) Two Challenges

1. Increased Complexity – Managing multiple services is harder than a monolith.
2. Network Communication Issues – Services communicate over the network, which can cause latency and failures.

(c) How Eureka and API Gateway Work Together

- Each microservice (UserService, OrderService, ProductService) registers itself with Netflix Eureka.
- The API Gateway uses Eureka to discover service locations dynamically.
- The React frontend sends all requests to a single gateway URL, such as:

  http://localhost:8080/api/orders

- The API Gateway forwards the request to the correct microservice using service names from Eureka.

This gives the frontend a single entry point instead of calling multiple service URLs directly.

Section D — Testing & Docker

Q17)

Ans:

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock
    private ProductRepository repo;
    @InjectMocks
    private ProductService service;
    @Test
    void testGetById() {
        Product product = new Product();
        product.setId(1L);
        product.setName("Laptop");
        when(repo.findById(1L)).thenReturn(Optional.of(product));
        Product result = service.getById(1L);
        assertEquals("Laptop", result.getName());
    }
}
```

Q18)

Ans:

@Mock is a Mockito annotation used for unit testing without loading the Spring context. It creates a fake object only for testing a class in isolation.

@MockBean is a Spring Boot Test annotation used when the Spring application context is loaded. It replaces a real Spring bean with a mock bean inside the application context.

Use @Mock for simple unit tests with Mockito.

Use @MockBean for integration or Spring Boot tests using @SpringBootTest.

Example using @MockBean with @SpringBootTest:

```
@SpringBootTest
public class ProductServiceTest {
    @MockBean
    private ProductRepository repo;
    @Autowired
    private ProductService service;
}
```

Section E — Frontend Integration (Bonus)

Q21)

Ans:

```
async function loadUsers() {
    const userList = document.getElementById("user-list");
    const error = document.getElementById("error");

    try {
        const response = await fetch('/api/users');
        if (!response.ok) {
            throw new Error("Failed to fetch users");
        }
        const users = await response.json();
        userList.innerHTML = "";
        users.forEach(user => {
            const li = document.createElement("li");
            li.textContent = user.name;
            userList.appendChild(li);
        });

    } catch (err) {
        error.textContent = err.message;
    }
}
```

Q22)
Ans:

(a) What is Wrong?

useEffect() has no dependency array, so it runs after every render. setUsers(data) causes a re-render, which again triggers useEffect(), creating an infinite loop.

(b) & (c) Fixed Code with Loading State

```
function UserList() {

    const [users, setUsers] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {

        fetch('/api/users')
            .then(res => res.json())
            .then(data => {
                setUsers(data);
                setLoading(false);
            });

    }, []); // dependency array added

    if (loading) {
        return <p>Loading...</p>;
    }

    return (
        <ul>
            {users.map(u => (
                <li key={u.id}>{u.name}</li>
            ))}
        </ul>
    );
}
```